

ASSIGNMENT 3

Université Jean Monnet – Saint-Étienne

2026/2027

This assignment contains only one exercise. Expected for submission: your completed `main.cpp` file as well as the written answers to the questions in step 1 and 4.

Exercise 1 — Image tweaker

To complete this exercise, you have to download the archive corresponding to your OS here: <https://grybouilli.github.io/introduction-to-programming.html>. The archive is also available on my website: <https://grybouilli.github.io/introduction-to-programming.html>. Decompress the archive and open the folder it contains in VSCode.

You will write a command-line program that lets the user manipulate images colors.

Invocation:

```
./img_tweaker <input_file> <output_file> <command> [args...]
```

Step 1 — Getting familiar with the provided library

Here is a correspondance table of types you can use (using one is equivalent to writing the other):

Type	Usable Alias
<code>std::vector<std::vector<std::vector<uint8_t>>></code>	<code>RGBImage</code>
<code>std::vector<std::vector<uint8_t>></code>	<code>BWImage</code>

Look into the file `libs/include/image_manips.hpp` and read the functions descriptions.

Which function can you use to read an rgb image from a file?

Which function can you use to save an rgb image into a file?

What does `swap_channels` do?

Does `swap_channels` modify the arguments it's given?

What does `convert_rgb_to_bw` do?

Does `convert_rgb_to_bw` modify the arguments it's given?

Step 2 — Writing the main function

The commands

The program takes a minimum of three arguments (`argc == 4`), but may take extra arguments depending on the [command](#) it is given. In the end, it will handle the following two commands:

Command name	Extra arguments	Program behavior
<code>tobw</code>	<i>no extra argument</i>	Converts the image read from <code>input_file</code> to black and white and saves the resulting image to <code>output_file</code>
<code>swap</code>	<code><channel1> <channel2></code> each one can take any value of <code>red green blue</code>	Swaps channels <code>channel1</code> and <code>channel2</code> of the image read from <code>input_file</code> and saves the resulting image to <code>output_file</code> . If one of the channel value given as input to the program is neither of <code>red green blue</code> , the program should output an error message.

Some invocation examples:

```
./img_tweaker input_file.png output_file.png tobw
./img_tweaker input_file.png output_file.png swap red green
./img_tweaker input_file.png output_file.png swap wrong channel
Error: channels can only take values in {red, green, blue}
```

To do

The `main` function has been written in the file `src/main.cpp`. It needs to be completed in the following manner:

1. In the `if` part where `command` is `"tobw"`:
 - Call the function `read_rgb_image` with `input_file_path` passed as argument. Store the result in a variable of type `RGBImage` named `image`.
 - Call the function `convert_rgb_to_bw` with `image` passed as argument. Store the result in a variable of type `BWImage` named `bw_image`.
 - Call the function `write_bw_image` with `output_file_path` and `bw_image` passed as arguments.
2. In the `else if` part where `command` is `"swap"`:

- After the inner `if`, call the function `read_rgb_image` with `input_file_path` passed as argument. Store the result in a variable of type `RGBImage` named `image`.
- Call the function `swap_channels` with `image`, `c1` and `c2` passed as arguments. Store the result in a new variable of type `RGBImage` named `image_with_swapped_channels`.
- Call the function `write_bw_image` with `output_file_path` and `image_with_swapped_channels` passed as arguments.

Step 3 — Compiling by hand

Your project should have the following layout:

```

src/
├── main.cpp
├── libs/
│   ├── include/
│   │   ├── image_manips.hpp
│   │   └── libimage_manips.so
└── test/
    └── rainbow.png

```

In the terminal, place yourself at the root of the project (you can do so by opening your project folder in VSCode and then opening a terminal in VSCode). Depending on your OS (Windows vs MacOS), run one of the following command line:

OS	Command to run
Windows	<code>clang++.exe -std=c++17 -I libs/include -Llibs -o img_tweaker.exe -limage_manips</code>
MacOS	<code>clang++ -std=c++17 -I libs/include -Llibs -o img_tweaker -limage_manips</code>

Table 1: Commands to build `img_tweaker`.

Test your program with the following lines:

To run on MacOS

```

export DYLD_LIBRARY_PATH=libs
./img_tweaker test/rainbow.png test/bw_rainbow.png tobw
./img_tweaker test/rainbow.png test/swapped_rainbow.png swap red green

```

To run on Windows

```

$env:PATH="./libs/;$env:PATH"
.\img_tweaker.exe test\rainbow.png test\bw_rainbow.png tobw
.\img_tweaker.exe test\rainbow.png test\swapped_rainbow.png swap red green

```

Step 4 — Building with cmake

Create a file named `CMakeLists.txt` at the root of the project. Copy and paste the following code inside it:

```
cmake_minimum_required(VERSION 3.14)
project(img_tweaker LANGUAGES CXX C)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_BUILD_TYPE Release)
FILE(GLOB PROJECT_SOURCES src/*.cpp)
add_executable(${PROJECT_NAME} ${PROJECT_SOURCES})
target_include_directories(${PROJECT_NAME} PRIVATE libs/include)
target_link_directories(${PROJECT_NAME} PRIVATE libs)
target_link_libraries(${PROJECT_NAME} PRIVATE image_manips)
```

In the terminal, place yourself at the root of the project. Using `cmake`, generate and build the project in a folder named `build`.

You final commands here:

Step 5 — Testing

Try invoking your compiled programs with the following commands:

To run on MacOS

```
export DYLD_LIBRARY_PATH=libs
./build/img_tweaker test/rainbow.png test/bw_rainbow.png tobw
./build/img_tweaker test/rainbow.png test/swapped_rainbow.png swap red green
```

To run on Windows

```
$env:PATH="./libs/;$env:PATH"
.\build\img_tweaker.exe test\rainbow.png test\bw_rainbow.png tobw
.\build\img_tweaker.exe test\rainbow.png test\swapped_rainbow.png swap red green
```